

ORUSD Token — Technical White Paper & Audit Reference

Public Technical Documentation · Osnias Clearing · www.osnias-clearing.com

Document status: public pre-audit technical architecture draft for developer, integrator and smart-contract review. The exact Solidity source and ABI are published separately as companion technical files.

AUDIT SCOPE PRINCIPLE

ORUSD is a restricted clearing-register token. Its security model is defined not by market composability, but by controlled issuance and destruction, EOA-only circulation, explicit oracle request boundaries, non-replayable external proofs and manager-gated settlement execution.

1. Technical Role of ORUSD

ORUSD is designed as a Sei-native clearing representation. The production token has one circulation purpose only: direct peer-to-peer transfer between user-controlled accounts on Sei. A clearing token is not a DeFi instrument. ORUSD is therefore intentionally excluded from DeFi, market, custody-contract and cross-chain use.

The token is intended to circulate directly between user-controlled accounts within the Sei network. Mint and burn operations are reserved to the ORUSD manager, either manually or after an authorized oracle has created a verifiable on-chain request.

CORE REGISTER INVARIANT

ORUSD must remain a clearing-register instrument. The protocol must not expose a native execution path that turns ORUSD into a freely routable DEX/AMM asset or that allows an oracle to mint or burn directly.

2. Deployment Model: Public Reference vs Production

Ethereum Sepolia — public reference and compliance test environment

ORUSD is publicly deployed on Ethereum Sepolia as the reference test implementation. This deployment is intended for developer accessibility, reproducibility, independent testing and inspection with standard Ethereum tooling. Sepolia is the public reference environment; it is not the canonical production clearing ledger.

Sei EVM — canonical production environment

The intended production deployment is Sei EVM. Sei is designated as the canonical Mainnet execution layer for ORUSD. The production contract address, privileged roles, oracle set, fee-recipient configuration and operational security controls must be independently published in the Osnias Clearing external registry.

CANONICAL STATE PRINCIPLE

The Ethereum Sepolia deployment is the public test reference. The future Sei EVM deployment is intended to become the production ORUSD state. Sepolia balances, Sepolia proofs and Sepolia administrative actions create no production rights on Sei.

2A. Sei Network Locality Invariant

Production ORUSD has a single network domain: Sei. No protocol mechanism may move, mirror, bridge, wrap, migrate or recreate production ORUSD on another blockchain.

NETWORK LOCALITY INVARIANT

No bridge. No cross-chain transfer. No OFT. No migration mechanism. Production ORUSD remains entirely within the Sei network.

The Ethereum Sepolia deployment is an isolated test instance only. It is not a bridge endpoint, wrapped representation, precursor balance or transferable claim on Sei ORUSD. Sepolia balances are test balances and are never migrated into the Sei production ledger.

Oracle references to an external event or identifier do not create cross-chain ORUSD. They are evidentiary inputs to a manager-controlled clearing decision; ORUSD issuance, destruction and circulation remain local to the canonical Sei deployment.

Public Reference Deployment Registry

Field	Ethereum Sepolia reference	Audit status
Network	Ethereum Sepolia	Deployed public test reference
Contract address	0xE971D62BE1d31867000A23D5536eEBF57f8D232d	Canonical Sepolia address for this document
Source reference	Orusd-testnet(1).sol	Exact Solidity source supplied after deployment
ABI reference	Supplied Sepolia ABI	Function/event surface consistent with reviewed source
Production status	Not Mainnet production	Separate Sei-native production deployment; no migration or balance continuity from Sepolia

SOURCE PUBLICATION PRINCIPLE

The exact Solidity source and ABI are published as companion technical files. This white paper therefore describes architecture, security boundaries and audit invariants rather than duplicating implementation code.

DEPLOYED REFERENCE STATUS

The address above identifies the public Sepolia test deployment reviewed by this document. It must not be represented as the future Sei Mainnet production address.

3. Token Identity and Numeric Model

Property	Current contract value	Audit interpretation
Solidity	^0.8.22	Compile reproducibly against the pinned OpenZeppelin dependency set.
Token name	Osnias Clearing	ERC-20 display name encoded by constructor.
Symbol	ORUSD	Public ticker used by the contract.
Technical decimals	6	On-chain precision; interfaces may display fewer decimals.
Protocol fee	15 bps = 0.15%	Deducted from the transferred amount.
Minimum gross P2P transfer	0.501000 ORUSD	Minimum gross value for an ordinary P2P transfer.
Oracle request validity	24 hours	Execution window from request creation time.

The public source also contains protocol-identification metadata. These technical labels must be distinguished from economic or legal claims: the code itself does not establish a bank deposit, fiat redemption promise or external USD reserve.

4. P2P Transfer Model

Ordinary P2P transfers follow a gross-debit / net-receipt model. The sender is debited by the amount submitted; the protocol fee is deducted from that amount and credited to the designated fee recipient. The recipient receives the net amount.

Example	Amount
---------	--------

Gross transfer request	1,000.000000 ORUSD
Protocol fee (0.15%)	1.500000 ORUSD
Recipient receives	998.500000 ORUSD
Sender total debit	1,000.000000 ORUSD

The designated fee recipient is exempt from the transfer fee when moving collected ORUSD, preventing recursive fee generation on protocol-fee movements.

TRANSFER INVARIANT

For an ordinary user transfer, sender debit = gross value; recipient credit + feeRecipient credit = gross value. No additional ORUSD is created by the fee mechanism.

5. EOA-Only Circulation and Anti-DEX Boundary

ORUSD deliberately disables the standard delegated-transfer and smart-contract pathways normally required by token markets and DeFi protocols. Ordinary circulation is restricted to direct P2P transfers between user-controlled accounts.

- EOA → EOA transfer: permitted, subject to minimum amount, fee rules and recipient blacklist.
- EOA → smart contract: rejected.
- Allowance-based delegated transfers: rejected.
- Delegated transfers by third parties: rejected.
- Issuance to smart-contract recipients: rejected.
- Burn: permitted only through manager-controlled paths.

ANTI-DEFI / MARKET ISOLATION INVARIANT

The intended design is stronger than the absence of a native swap feature. ORUSD must not expose the delegated-transfer, contract-recipient or interoperability pathways normally required by escrows, vaults, AMMs, DEX routers, staking, farming, lending, liquidity pools or other DeFi integrations.

5A. Market Isolation

The intended production model does not support swap markets, automated market makers, decentralized exchanges, centralized exchange markets, liquidity pools, farming, token wrapping or smart-contract routing for ORUSD.

CLEARING-ONLY PRINCIPLE

No swap. No AMM. No DEX. No protocol-supported CEX. No escrow. No vault. No staking. No farming. No lending. No smart-contract circulation. ORUSD is a P2P clearing token on Sei, not a DeFi or market token.

The smart contract can enforce the absence of smart-contract routing and delegated-transfer mechanisms. A blockchain cannot reliably determine whether an ordinary EOA is controlled by an individual or by an off-chain custodian; therefore the CEX prohibition is also an Osnias Clearing protocol and integration rule. No CEX listing, market or custodial integration is authorized or supported by the protocol.

6. Restricted Recipient Domain

The contract maintains a restricted-recipient mechanism for infrastructure addresses that must not receive ordinary ORUSD transfers. The initial manager and GasPool are restricted at deployment. Authorized oracle addresses are also restricted from receiving ORUSD, and this receiving restriction remains after oracle authorization is removed.

- Initial manager: cannot receive ORUSD through the transfer path.
- GasPool: cannot receive ORUSD.
- Authorized oracle: cannot receive ORUSD.
- Former oracle: remains blacklisted after de-authorization.
- Protocol fee recipient: intentionally eligible to receive ORUSD so that transfer fees can be collected.

ROLE-SEPARATION PRINCIPLE

Operational infrastructure addresses that validate, manage or sponsor the system should not accumulate user clearing balances merely because they hold privileged roles.

7. Oracle Request Architecture

An authorized oracle does not mint or burn ORUSD. Its authority is limited to creating a request after observing or validating an external event. The request is written on-chain and becomes inspectable before any supply change occurs.

Each oracle request records the information required to identify its nature, status, submitting oracle, affected account, amount, external source reference and creation time. Exact field names and encoding are documented in the public Solidity source and ABI.

Request field	Purpose
Request category	MINT or BURN
Lifecycle status	Pending, executed or rejected
Submitting oracle	Authorized source of the request
Affected account	EOA whose ORUSD balance may change
Amount	Requested ORUSD quantity
External evidence source	Identifier of the observed clearing event or record; not a token network destination
External reference	Reference to the external clearing evidence; does not represent bridged ORUSD
Creation time	Used to enforce the 24-hour validity window

ORACLE AUTHORITY INVARIANT

An oracle may attest and request. It may not settle. Only the ORUSD manager can execute or reject an oracle request.

8. Clearing-Proof Uniqueness and Anti-Replay

Each clearing proof reference can be consumed only once. The contract stores a unique proof identifier for the referenced external event or record, preventing a second request from reusing the same evidence. This evidentiary reference does not represent a bridge or cross-chain token transfer.

Each request also receives a unique on-chain identifier and sequential audit trail. The exact identifier derivation is intentionally left to the published source code rather than reproduced in this white paper.

ANTI-REPLAY INVARIANT

A given external clearing reference can create at most one ORUSD oracle request. Rejection does not free that proof for reuse.

9. Request Lifecycle and Settlement

A valid oracle request begins in a pending state. The manager may execute it within the 24-hour validity period or reject it. Expired requests cannot be executed.

- MINT request: valid execution increases the designated account balance and total supply by the approved amount.
- BURN request: valid execution reduces the designated account balance and total supply by the approved amount, subject to sufficient balance.
- Before mint/burn execution, status is set to EXECUTED, so the stored request cannot be replayed.
- A rejected request is permanently marked REJECTED.
- An expired request remains part of the audit history but cannot be settled through the normal execution path.

SETTLEMENT FINALITY PRINCIPLE

Every supply-changing oracle operation must correspond to one identifiable request and one terminal status transition.

10. Manual Manager Mint and Burn

The deployed Sepolia reference includes manager-controlled manual issuance and destruction functions. These are part of the current test-reference implementation and constitute the strongest privileged supply authority in the contract.

Manual issuance is limited to eligible non-zero EOA recipients and positive amounts. Manual destruction is manager-controlled and applies to a specified account, subject to the contract conditions.

AUDIT ATTENTION — PRIVILEGED SUPPLY

The manager can currently mint to an eligible EOA and burn from an arbitrary ORUSD account. Production governance and operational-security controls must therefore treat the manager key as a critical supply authority.

11. Administrative Authority Matrix

Action	Oracle	Manager / owner	User
Submit oracle request	Yes, if authorized	No special need	No
Execute oracle request	No	Yes	No
Reject oracle request	No	Yes	No
Manual issuance	No	Yes	No
Manual destruction	No	Yes	No
Authorize / remove oracle	No	Yes	No
Change protocol fee recipient	No	Yes	No
P2P transfer	Blocked as recipient role; normal sending depends on balance	Normal token restrictions apply	Yes
Delegated transfer	No	No	No

SEPARATION OF DUTIES

Oracle observation and manager settlement are separate trust domains. A compromised oracle alone must not be sufficient to create or destroy ORUSD.

12. ORUSD / OSNIAS Economic Firewall

The OSNIAS technical white paper defines OSNIAS as the governance and strategic-reserve asset and requires strict separation from clearing-register tokens. ORUSD is one such clearing-register domain and therefore sits on the opposite side of that firewall.

- No protocol-native ORUSD ↔ OSNIAS swap.
- No protocol-authorized ORUSD / OSNIAS liquidity pair.
- No ORUSD / OSNIAS farming relationship created by Osnias Clearing.
- No OSNIAS collateral dependency for ORUSD issuance or settlement.
- No administrative role in OSNIAS should automatically grant ORUSD administrative authority.

CROSS-DOMAIN SECURITY INVARIANT

Compromise of OSNIAS governance or OSNIAS privileged administration must not, by design, grant control over ORUSD mint, burn, oracle or fee-recipient authority.

13. Supply and Accounting Invariants

Invariant	Required property	Manager override
No hidden mint path	Supply increases only through explicit manager issuance or execution of a valid MINT request.	Not outside explicit functions

No hidden burn path	Supply decreases only through explicit manager destruction or execution of a valid BURN request.	Not outside explicit functions
Oracle non-custody	Oracle cannot directly move, mint or burn user ORUSD.	Not permitted
Proof uniqueness	One external proof pair can be consumed once.	Not permitted
P2P conservation	Ordinary transfer redistributes existing ORUSD between recipient and feeRecipient.	Not permitted
No delegated transfer	Allowance-based third-party transfer mechanisms remain disabled.	Not permitted by current implementation
EOA-only recipient rule	Ordinary transfers to deployed contracts revert.	Not permitted by current code
Sei locality	Production ORUSD exists and circulates only on Sei; no bridge, OFT, wrapping or migration path.	Not permitted
Market isolation	No protocol path for swap, AMM, DEX, protocol-supported CEX market or liquidity integration.	Not permitted

14. Security Validation Pipeline — Clearing, Not DeFi

ORUSD is intentionally compact because its purpose is clearing, not DeFi. Security review must therefore verify both the correctness of the permitted clearing paths and the absence of prohibited DeFi or market surfaces. Static analysis, including Slither, is used to map exposed functions, privilege boundaries, external calls and token-transfer pathways so that unintended smart-contract, escrow, router, liquidity or cross-chain integration paths can be identified. Slither is not an audit and does not certify DeFi compatibility; for ORUSD, DeFi compatibility would contradict the intended architecture.

CLEARING SECURITY PRINCIPLE

A successful ORUSD security review does not ask whether the token can integrate safely with DeFi. It asks whether DeFi integration is technically unavailable by design, while direct P2P clearing on Sei remains reliable and auditable.

Security objective	Primary method	Audit interpretation
Public interface exposure	Slither + manual review	Map all externally callable state-changing paths against the published ABI.
Manager privilege boundaries	Slither + unit tests	Verify every supply/admin function is restricted as intended.
Oracle authorization	Unit + fuzz tests	Unauthorized oracle request creation must always fail.
Proof replay	Invariant tests	The same external proof reference cannot produce a second request.
Request expiry	Time-based tests	Execution after 24h must revert; rejection behavior remains deterministic.
Mint/burn accounting	Invariant tests	Supply change must equal the authorized amount.
Fee conservation	Property tests	sender debit = recipient net + fee.
EOA-only transfer	Integration tests	Transfers to contracts, escrow contracts, vaults and router pathways must revert.
Delegated-transfer lockout	Unit tests	No allowance-based transfer route may be available.

OSNIAS firewall	Multi-contract review	No protocol-controlled conversion or shared privileged authority.
Network locality	Slither + source + integration review	Verify absence of bridge, OFT, cross-chain messaging, wrapping, mirroring and migration mechanisms.
DeFi and market isolation	Slither + source + integration review	Verify no smart-contract routing, escrow, vault, AMM, DEX, staking, farming, lending, LP or protocol-supported market path exists.
Clearing-only architecture	Manual review + invariant tests	Confirm that every permitted token path serves issuance, burn, fee collection or direct P2P clearing on Sei; no DeFi function is required for normal operation.

15. Recommended Pre-Audit Sequence

1. Reproducible Solidity compilation with pinned OpenZeppelin version.
2. Slither static analysis focused on clearing invariants and verification of absent DeFi / smart-contract integration surfaces.
3. Unit tests for mint, burn, fee, blacklist and ownership controls.
4. Fuzz and invariant testing for supply conservation and proof replay.
5. Ethereum Sepolia deployment and public integration tests — reference deployment completed at 0xE971D62BE1d31867000A23D5536eEBF57f8D232d; continue functional and invariant testing against this deployed instance.
6. Oracle lifecycle tests using realistic clearing-reference datasets, including replay and expiry cases.
7. Wallet tests for direct P2P transfers between Sei user-controlled accounts, without smart-contract routing.
8. Sei EVM deployment rehearsal with production-like addresses and multisig configuration.
9. Manual security review against the invariants in this white paper.
10. Final Sei Mainnet deployment review and external registry publication.

16. Mainnet Privileged-Key Security

The public Sepolia reference deployment uses the current simplified administrative model suitable for development and testability. The production Sei deployment is intended to replace single-key privileged administration with a stronger operational-security architecture for critical manager authority.

- Production manager authority: multisignature control rather than a single EOA.
- Cold / emergency signer separation: recovery keys stored outside ordinary operational devices.
- Oracle separation: oracle signing infrastructure must be independent from manager settlement keys.
- Fee wallet separation: feeRecipient should not be the same key as the oracle or privileged manager unless explicitly justified.
- External registry: authoritative production addresses, roles and recovery procedures maintained outside the token contract.
- Incident procedure: documented oracle removal, ownership transfer and emergency operational response.

MAINNET SECURITY PRINCIPLE

A single compromised key should not be sufficient to both fabricate an external attestation and execute the corresponding ORUSD supply change.

17. Deployed Sepolia Source and Mainnet Delta

Implementation Status Classification

Category	Meaning in this document	Examples
IMPLEMENTED — Sepolia	Present in the exact supplied source and exposed by the supplied ABI.	Six decimals; fee logic; EOA-only recipient restriction; oracle-request workflow; manager-controlled supply operations; anti-replay; delegated transfers disabled.
CONFIGURATION — Sepolia	Deployment-specific addresses or comments that identify the current test environment.	Sepolia contract address; manager, protocol fee recipient and GasPool addresses.
REQUIRED — Sei Mainnet	Production-security requirement or deployment rule not claimed to be enforced by the current Sepolia bytecode.	Sei-only production deployment; no bridge/OFT/migration; multisig manager; cold-key separation; production registry; incident/recovery procedures.
RECOMMENDED HARDENING	Security improvement to evaluate before Mainnet; not represented as currently implemented.	Ownership migration handling; ownership-renunciation policy; fee-recipient / restricted-recipient validation.

The exact Solidity source supplied for the Sepolia deployment is the technical implementation reference for this document. Its public behavior is consistent with the supplied ABI and with the architectural properties described here. Detailed function signatures, storage names, error selectors and implementation expressions are intentionally not reproduced in this white paper because the source and ABI are published alongside it.

- Clean legacy source comments that still identify earlier Polygon test or production environments; the current public reference deployment is Ethereum Sepolia.
- Update the GasPool documentation for each deployment environment. Sepolia uses ETH for gas; the future Sei Mainnet deployment uses SEI as the native gas asset.
- Replace deployment-specific manager, fee-recipient and GasPool addresses for the production Sei deployment, and record the final values in the external registry.
- Verify that deployment-specific addresses satisfy the EOA restrictions required by the contract.
- Regenerate a correct flattened ORUSD source for explorer verification when required. The previously supplied flat file contains OSNIAS code and is not a valid flatten of ORUSD.
- Re-evaluate ownership-transfer behavior so that privileged-role migration and receiving restrictions remain coherent after a manager transition.
- Define the production policy for ownership renunciation and ensure that irreversible loss of administrative execution authority cannot occur accidentally.
- Validate fee-recipient configuration against the restricted-recipient rules so an administrative configuration change cannot block fee-bearing transfers.

PRE-MAINNET REVIEW QUESTION

Can the audited Sepolia reference implementation be independently deployed on Sei production without introducing bridge, migration, OFT or cross-chain functionality, while preserving all documented P2P, market-isolation and security invariants?

18. Auditor Acceptance Checklist

- No oracle can directly create or destroy supply.
- Only the manager can execute or reject oracle requests.
- Every executed oracle request was previously PENDING and unexpired.
- Every external proof pair is single-use.
- Manual mint/burn paths are fully enumerated and privilege-gated.
- Ordinary P2P transfer accounting conserves token units.

Production ORUSD has no bridge, OFT, wrapping, cross-chain transfer or migration mechanism.

The Sepolia test instance has no balance continuity or migration path into Sei Mainnet.

No protocol-supported swap, AMM, DEX, CEX market, escrow, vault, staking, farming, lending, liquidity-pool or smart-contract circulation path exists.

- Delegated-transfer pathways cannot be used.
- Ordinary transfers to smart contracts revert.
- Blacklisted infrastructure addresses cannot receive ORUSD.
- Fee-recipient behavior cannot create recursive fees or a transfer deadlock.
- Ownership transfer, renunciation and role migration have explicit production procedures.
- ORUSD and OSNIAS have no protocol-native liquidity, swap, collateral or administrative-control bridge.
- The Sepolia reference address and the future Sei production address are clearly distinguished in public documentation, registry records and explorer verification.

FINAL SECURITY OBJECTIVE

ORUSD should do one thing and do it well: provide direct peer-to-peer clearing transfers between user-controlled accounts on Sei. Production ORUSD must remain local to Sei, non-bridged, non-OFT, non-migratable, non-swappable, non-AMM, non-DEX, non-protocol-supported-CEX, non-escrow, non-vault, non-staking, non-farming, non-lending and unavailable to ordinary smart-contract routing.

PREPARED BY — Denis Bouzon · ORCID 0009-0007-8894-8902 · Osnias Clearing · www.osnias-clearing.com

Document revision: v4 — Sei-locality edition. Network locality and market isolation are first-order protocol invariants; exact Solidity source and ABI are published separately alongside this document.